COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/2/2026	**DATE SUBMITTED:**	9/2/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 7.2 **Algorithm Analysis: Best-Case and Worst-Case Time Complexity**

1. PROCEDURE OUTPUT

Instructions: Analyze each algorithm carefully. For every item, determine the following:

1. **Best-Case Time Complexity**
2. **Worst-Case Time Complexity**
3. **Brief explanation** of how you arrived at your answers.

Use asymptotic notation such as **O(1), O(log n), O(n), O(n log n), O(n²)**, etc.

1. Finding an Element

```
C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }
}
```

Answer:

1. Best Case: O(1)
The target is in the first element (i = 0). Therefore, one comparison takes place in the loop.

2. Worst Case: O(n)
The target is in the last element or not in the array at all. In this case, the algorithm goes through all n elements.

3. Explanation: The number of operations is dependent on the location of the target. If the input is in the first element, it will stop; otherwise, it will keep iterating until it reaches an element that is equal to the target or reaches the end of the array.

2. Printing All Elements

```
C/C++
for (int i = 0; i < n; i++)
{
    cout << arr[i] << endl;
}
```

Answer:

1. Best Case: O(n)
The loop iterates for all elements of the array and prints them out. As there are n elements, the loop executes n times.

2. Worst Case: O(n)
The loop still has to go through every element and print each one, so it also runs n times.

3. Explanation:
Since the program has to access and print each and every item in the array, the steps will increase linearly O(n) with the size of the array. Which makes both cases O(n).

3. Nested Loops

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cout << i << " " << j << endl;
    }
}
```

Answer:

1. Best Case: $O(n^2)$
The outer loop executes n times, and for each outer loop, the inner loop too executes n times. This means that

the total number of iterations will be (n)(n) = n^2.

2. Worst Case: O(n^2)
The same thing happens every time because both loops always run n times. There is no condition that allows either loop to stop early.

3. Explanation:
Since one loop is inside another loop, we multiply their work: (n)(n) = n^2

4. Conditional Processing

```
C/C++
if (n > 100)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i];
    }
}
else
{
    cout << "Small input";
}
```

Answer:

1. Best Case: O(1)
If n < 100, then the else part executes, resulting in only "Small input" being outputted. It requires just one step, hence it is O(1).

2. Worst Case: O(n)
When n is greater than 100, the for loop executes n times in order to print all the items. Hence, it is O(n).

3. Explanation:
If n is small, it just prints a message ("Small input") that is O(1). But when n is big, it runs through all n elements, hence O(n).

5. Fixed Inner Loop

```C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < 10; j++)
    {
        cout << i << j;
    }
}
```

Answer:

1. Best Case: O(n)
The outer loop takes n iterations. The inner loop will take 10 iterations, which is a constant every time you run the loop. So the total amount of work is n x 10 which concludes it is O(n).

2. Worst Case:O(n)
This will be the same case since the inner loop runs 10 times in each iteration of the outer loop. Similarly the total work is equal to n x 10, O(n) again.

3. Explanation:
The inner loop executes for 10 iterations always, but the outer loop depends upon n. Hence, the complexity will be O(n).

6. Two Consecutive Loops

```C/C++
for (int i = 0; i < n; i++)
{
    cout << arr[i];
}

for (int j = 0; j < n; j++)
{
    cout << arr[j] * 2;
}
```

Answer:

1. Best Case: O(n)
The first loop iterates n times and the second loop iterates n times. Combined, they perform n + n = 2n operations, resulting in O(n).

2. Worst Case:O(n)
Both loops always run n times, so the total is still 2n, which is O(n).

3. Explanation:
The loops are not nested within each other, they are executed sequentially; so we sum them up to get n + n = 2n and if we write it in standard form results in O(n).

7. Dividing by Two

```C/C++
int x = n;

while (x > 1)
{
    x = x / 2;
}
```

Answer:

1. Best Case: O(log n)
x is divided by 2 multiple times until the condition is met where x = 1. This number of iterations increases gradually as the value of n becomes larger, which in this case we can conclude O(log n).

2. Worst Case: O(log n)
The loop will continue to divide x by 2 till it becomes 1. when n is large, because the loop has to perform more divisions before reaching 1.

3. Explanation:
Each iteration of the loop, x becomes half of its previous value. Since the loop keeps halving the number, its complexity will be O(log n).

8. Nested Loop With Different Limits

```C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << i << j;
    }
}
```

Answer:

1. Best Case: O(n²)
There is only one cycle that repeats n times, but as i increases, the inner loop is repeated many times. This makes the number of operations gets close to n².

2. Worst Case: O(n²)
The inner loop is iterated many times when i is very large. So, the number of iterations is around n²/2, and that is considered O(n²).

3. Explanation:
The inner loop gets longer as the outer loop progresses. That makes the complexity O(n²).

9. Early Termination

```C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        cout << "Found";
        break;
    }
}
```

Answer:

1. Best Case: O(1)
The value being searched for is the first element, so the program finds it immediately.

2. Worst Case:O(n)
The value being searched for is the last element or is not in the array, so the program checks every element.

3. Explanation:
The loop can stop early when it finds the target. If it finds it right away, it is O(1). If it has to check everything, it is O(n).

10. Two-Level Search

```C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (arr[i] == target)
        {
            break;
        }
    }
}
```

Answer:

1. Best Case: O(n)
The required value is found straight away. But the outer loop executes n times, thus resulting in a complexity of O(n).

2. Worst Case: $O(n^2)$
For each loop iteration of the outer loop, the inner loop executes n times. Therefore, n x n = n^2, and thus the time complexity is $O(n^2)$.

3. Explanation:
There are two loops, but the inner loop may stop early in case the value is found. The best case scenario is that the inner loop stops as soon as possible, but the outer loop still executes n times. In the worst case, both loops execute n times.

11. Linear Loop With Logarithmic Inner Work

```
C/C++
for (int i = 0; i < n; i++)
{
    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Answer:

1. Best Case: O(n log n)
The for loop executes n times. For each execution of the for loop, the while loop continues dividing x by 2, which takes O(log n) iterations. Hence, n x log n = O(n log n).

2. Worst Case: O(n log n)
This procedure is the same for all iterations of the for loop. The while loop has a cost of O(log n), and the for loop iterates n times. So, the overall complexity is O(n log n).

3. Explanation:
The first loop executes n times and at each iteration, the second loop divides by 2. We thus get the result of n × log n which is O(n log n).

12. Conditional Nested Loop

```
C/C++
if (arr[0] == target)
{
    cout << "Found";
}
else
{
```

```
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << arr[i] + arr[j];
        }
    }
}
```

Answer:

1. Best Case: O(1)
If the value of the first element is the same as the search value, then the program only prints “Found” and stops straightaway.

2. Worst Case: O(n²)
But if the value of the first element is not equal to the value required, then both the loops are executed, and each loop is executed n number of times, which means n x n = n².

3. Explanation:
The program searches for the element once. If it finds the element it requires, then it stops there itself, which takes only 1 time to execute, meaning O(1). Otherwise, it continues for two loops, and that takes O(n²) time.

13. Finding the Maximum

```C/C++
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Answer:

1. Best Case: O(n)
The program must go through each element of the array in order to ensure that no other greater value exists. Therefore, the loop iterates n-1 times, making its complexity O(n).

2. Worst Case: O(n)
The program also checks every element to find the largest value. Even if max changes many times, the loop still runs n - 1 times, so it is O(n).

3. Explanation:
To get the maximum number, the program needs to go through all the elements. Since it always processes the whole array, its best and worst case are O(n).

14. Three Nested Loops

```
C/C++
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        for (int k = 0; k < n; k++)
        {
            cout << i + j + k;
        }
    }
}
```

Answer:

1. Best Case: O(n^3)
The three nested loops will all execute n times. Because there is no condition to stop them early, their overall effort will be n x n x n = n^3.

2. Worst Case: O(n^3)
All three loops still run n times, so the total work remains n x n x n = n^3.

3. Explanation:
We have three nested loops, and each loop runs n times. We can multiply them and that will be n x n x n = n^3. In both cases, O(n^3).

15. Comprehensive Analysis

```C/C++
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        return i;
    }

    int x = n;

    while (x > 1)
    {
        x = x / 2;
    }
}
```

Answer:

1. Best Case: O(1)
The search value exists in the first array element. Therefore, the program directly returns the value without running the while loop.

2. Worst Case: O(n log n)
If the element is not present, the for loop will execute for n iterations, and in every iteration, the while loop will divide x by 2, taking O(log n) operations. Hence, n x log n = O(n log n).

3. Explanation:
It is accessed right away; hence its value is O(1) (Best Case). All the n elements are traversed, and within each element, there is a logarithmic loop, leading to O(n log n) (Worst-Case).

Conclusion:

During this seatwork, I got a glimpse of how one can examine the time complexity of different codes by examining how their loops and conditionals behave. I learned that simple loops can be O(n), nested loops can result in either $O(n^2)$ or $O(n^3)$, while loops that halve the values can be O(log n). Moreover, I learned that there can be differences in the best and worst cases because of the ability or inability of the code to end earlier without processing the entire input. Generally speaking, this seatwork taught me how the size of the input influences the efficiency of an algorithm. Apart from that, I should admit that I have to go back to my general

math course in order to fully comprehend it; however, apart from that, it was quite a useful seatwork to start with given my total lack of knowledge.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini (Google Overview) and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, explanation of concepts and idea generation.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to correct grammar and help identify misconceptions.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.